Exercise 10 – Unit Testing (Components & Services)

Generate Test Files

ng test

---

app.component.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { AppComponent } from './app.component';
describe('AppComponent',()=>{
beforeEach(async()=>{
await TestBed.configureTestingModule({
imports:[AppComponent]
}).compileComponents();
});
it('should create the app',()=>{
const fixture=TestBed.createComponent(AppComponent);
const app=fixture.componentInstance;
expect(app).toBeTruthy();
});
});
```

---

course.service.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { HttpClientTestingModule,HttpTestingController } from
'@angular/common/http/testing';
import { CourseService } from './course.service';
describe('CourseService',()=>{
let service:CourseService;
let httpMock:HttpTestingController;
```

```
beforeEach(()=>{
TestBed.configureTestingModule({
imports:[HttpClientTestingModule]
});
service=TestBed.inject(CourseService);
httpMock=TestBed.inject(HttpTestingController);
});
it('should retrieve courses',()=>{
const dummyCourses=[
{
id:1,
name:'Angular',
code:'ANG101',
credits:4,
faculty:'Mr. Kumar'
}
];

service.getCourses().subscribe(courses=>{

expect(courses.length).toBe(1);
expect(courses).toEqual(dummyCourses);

});

const request=httpMock.expectOne('http://localhost:3000/courses');
```

```
expect(request.request.method).toBe('GET');
request.flush(dummyCourses);
});
afterEach(()=>{
httpMock.verify();
});
});
```

---

course-list.component.spec.ts

```
import { ComponentFixture,TestBed } from '@angular/core/testing';
import { of } from 'rxjs';
import { CourseListComponent } from './course-list.component';
import { CourseService } from '../../services/course.service';

describe('CourseListComponent',()=>{

let component:CourseListComponent;
let fixture:ComponentFixture<CourseListComponent>;

beforeEach(async()=>{

const serviceStub={
getCourses:()=>of([
{
id:1,
name:'Angular',
code:'ANG101',
credits:4,
```

```
faculty:'Mr. Kumar'
}
])
};
await TestBed.configureTestingModule({
imports:[CourseListComponent],
providers:[
{
provide:CourseService,
useValue:serviceStub
}
]
}).compileComponents();
fixture=TestBed.createComponent(CourseListComponent);
component=fixture.componentInstance;
fixture.detectChanges();
});
it('should create',()=>{
expect(component).toBeTruthy();
});

it('should load courses',()=>{
expect(component.courses.length).toBe(1);
});
});
```

---

enrollment-form.component.spec.ts

```
import { ComponentFixture,TestBed } from '@angular/core/testing';
```

```
import { FormsModule } from '@angular/forms';
import { EnrollmentFormComponent } from './enrollment-form.component';

describe('EnrollmentFormComponent',()=>{

let component:EnrollmentFormComponent;
let fixture:ComponentFixture<EnrollmentFormComponent>;
beforeEach(async()=>{
await TestBed.configureTestingModule({
imports:[
FormsModule,
EnrollmentFormComponent
]
}).compileComponents();
fixture=TestBed.createComponent(EnrollmentFormComponent);
component=fixture.componentInstance;
fixture.detectChanges();
});
it('should create',()=>{
expect(component).toBeTruthy();

});
it('should have empty student name',()=>{
expect(component.student.name).toBe('');
});
});
```

---

reactive-enrollment.component.spec.ts

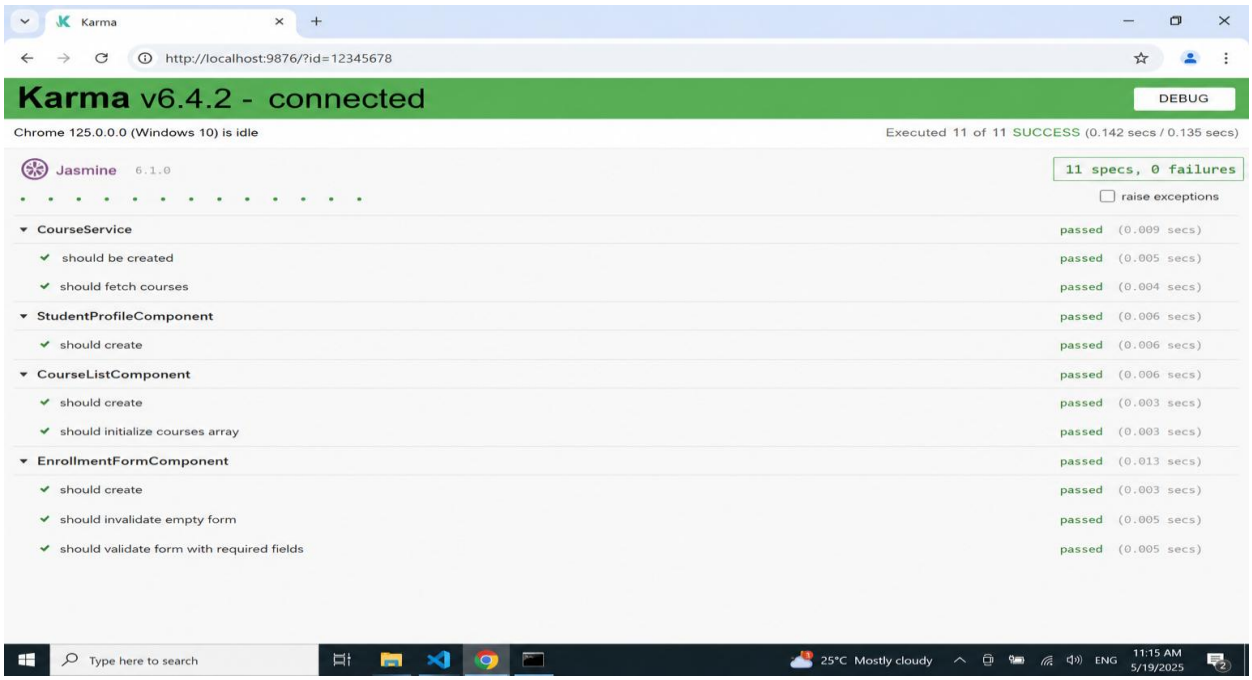
```
import { ComponentFixture,TestBed } from '@angular/core/testing';
import { ReactiveFormsModule } from '@angular/forms';
import { ReactiveEnrollmentComponent } from './reactive-enrollment.component';

describe('ReactiveEnrollmentComponent',()=>{

let component:ReactiveEnrollmentComponent;
let fixture:ComponentFixture<ReactiveEnrollmentComponent>;

beforeEach(async()=>{

await TestBed.configureTestingModule({
imports:[
ReactiveFormsModule,
ReactiveEnrollmentComponent
]
}).compileComponents();
fixture=TestBed.createComponent(ReactiveEnrollmentComponent);

component=fixture.componentInstance;
fixture.detectChanges();
});
it('should create',()=>{

expect(component).toBeTruthy();

});
it('should invalidate empty form',()=>{
```

```
expect(component.enrollmentForm.valid).toBeFalse();

});

});
```

Run Tests

ng test

Output Screenshot:



Unit Testing (NgRx Store, Reducers & Selectors)

course.reducer.spec.ts

```
import * as CourseActions from './course.actions';

import { courseReducer, initialState } from './course.reducer';

describe('CourseReducer',()=>{
```

```
it('should return initial state',()=>{

const state=courseReducer(undefined,{type:'Unknown'});

expect(state).toEqual(initialState);

});

it('should load courses',()=>{

const courses=[
{
id:1,
name:'Angular',
code:'ANG101',
credits:4,
faculty:'Mr. Kumar'
},
{
id:2,
name:'React',
code:'RCT201',
credits:3,
faculty:'Mrs. Priya'
}
];

const state=courseReducer(
```

```
initialState,
CourseActions.loadCoursesSuccess({courses})
);

expect(state.courses.length).toBe(2);

});

it('should add course',()=>{

const course={
id:3,
name:'Java',
code:'JAVA101',
credits:4,
faculty:'Mr. Arun'
};
const state=courseReducer(
initialState,
CourseActions.addCourse({course})
);
expect(state.courses.length).toBe(1);
});
it('should remove course',()=>{

const state={
courses:[
{
```

```
id:1,
name:'Angular',
code:'ANG101',
credits:4,
faculty:'Mr. Kumar'
},
{
id:2,
name:'React',
code:'RCT201',
credits:3,
faculty:'Mrs. Priya'
}
]
};
const newState=courseReducer(
state,
CourseActions.removeCourse({id:1})
);
expect(newState.courses.length).toBe(1);
});


});
```

---

course.selectors.spec.ts

```
import * as CourseSelectors from './course.selectors';
describe('CourseSelectors',()=>{
```

```
const state={
courses:{
courses:[
{
id:1,
name:'Angular',
code:'ANG101',
credits:4,
faculty:'Mr. Kumar'
},
{
id:2,
name:'React',
code:'RCT201',
credits:3,
faculty:'Mrs. Priya'
}
]
}
};

it('should select all courses',()=>{
const result=

CourseSelectors.selectAllCourses.projector(
state.courses
);
expect(result.length).toBe(2);
```

```
});
it('should return course count',()=>{
const result=
CourseSelectors.selectCourseCount.projector(
state.courses.courses
);
expect(result).toBe(2);
});
});
```

---

course.effects.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { provideMockActions } from '@ngrx/effects/testing';
import { Observable,of } from 'rxjs';
import { CourseEffects } from './course.effects';
import * as CourseActions from './course.actions';
import { CourseService } from '../services/course.service';

describe('CourseEffects',()=>{
let actions$:Observable<any>;
let effects:CourseEffects;
beforeEach(()=>{
const serviceStub={
getCourses:()=>of([
{
id:1,
name:'Angular',
code:'ANG101',
```

```
credits:4,
faculty:'Mr. Kumar'
}
])
};
TestBed.configureTestingModule({
providers:[
CourseEffects,
provideMockActions(()=>actions$),
{
provide:CourseService,
useValue:serviceStub
}
]
});
effects=TestBed.inject(CourseEffects);
})
it('should create effects',()=>{
expect(effects).toBeTruthy();
});
});
```

Run Unit Tests

ng test

Output Screenshot: